

CpfApi — Consulta de CPF e CNPJ

API REST em C# (.NET 8) para consulta de dados de **CPF** e **CNPJ** na Receita Federal, com autenticação via Bearer Token.

Tecnologias

Tecnologia	Versão	Uso
C# / .NET	8.0	Linguagem e runtime
ASP.NET Core Minimal API	8.0	Framework HTTP
ReceitaWS	—	API externa de consulta
Docker	24+	Containerização
xUnit	2.7	Testes unitários e de integração

Quickstart com Docker

```
# 1. Configurar variáveis de ambiente
cp CpfApi/.env.example CpfApi/.env
# Edite CpfApi/.env e defina ApiToken com um valor seguro

# 2. Build e inicialização (a partir da raiz do repositório)
docker compose up --build -d

# 3. Verificar saúde
curl http://localhost:8081/health

# 4. Consultar CPF
curl -H "Authorization: Bearer supersecret123" \
  http://localhost:8081/consulta/cpf/529.982.247-25

# 5. Consultar CNPJ
curl -H "Authorization: Bearer supersecret123" \
  "http://localhost:8081/consulta/cnpj/11.222.333/0001-81"
```

A API fica exposta na porta **8081** pelo docker-compose (mapeada para 8080 no container).

Quickstart local (sem Docker)

```
cd CpfApi

# Configurar variáveis
cp .env.example .env
# Edite .env

# Rodar
dotnet run
```

O servidor inicia na porta **8080** por padrão.

Estrutura do Projeto

```
CpfApi/
├─ Program.cs                # Entry point / registro de rotas
├─ CpfApi.csproj
├─ appsettings.json
├─ .env / .env.example
├─ Dockerfile
├─ Models/
│  ├─ CpfResponse.cs         # DTO de resposta CPF
│  ├─ CnpjResponse.cs        # DTO de resposta CNPJ
│  ├─ ReceitaWsResponse.cs   # DTO ReceitaWS (CPF)
│  └─ ReceitaWsCnpjResponse.cs # DTO ReceitaWS (CNPJ)
├─ Services/
│  ├─ ICpfService.cs / CpfService.cs
│  └─ ICnpjService.cs / CnpjService.cs
├─ Validators/
│  ├─ CpfValidator.cs        # Validação mod-11 (CPF)
│  └─ CnpjValidator.cs       # Validação mod-11 (CNPJ)
├─ Middleware/
│  └─ BearerAuthMiddleware.cs # Autenticação Bearer Token
└─ docs/
   ├─ API.md                 # Endpoints, exemplos, tabela de erros
   └─ TESTES.md              # Suite de testes e cobertura
```

Documentação

- [Documentação da API](#) — endpoints, exemplos curl/Postman, tabela de erros
- [Documentação de Testes](#) — suite xUnit, cobertura, como rodar

Rodando os Testes

```
cd CpfApi.Tests
dotnet test
```

60 testes — todos passando. Nenhuma dependência externa necessária (HttpClient e serviços são mockados).

Variáveis de Ambiente

Variável	Obrigatória	Padrão	Descrição
ApiToken	Sim	—	Bearer Token para autenticação da API
ReceitaWsToken	Não	—	Token da assinatura ReceitaWS (CPF)
ReceitaWsUrl	Não	<code>https:// www.receitaws.com.br/v1/ cpf</code>	URL base da API externa (CPF)
ReceitaWsCnpjToken	Não	—	Token da assinatura ReceitaWS (CNPJ)
ReceitaWsCnpjUrl	Não	<code>https:// www.receitaws.com.br/v1/ cnpj</code>	URL base da API externa (CNPJ)
ASPNETCORE_URLS	Não	<code>http://+:8080</code>	Porta do servidor (configurado no Dockerfile)

Documentação da API

Visão Geral

API REST em C# (.NET 8) para consulta de **CPF** e **CNPJ** na Receita Federal via [ReceitaWS](#). A API valida os documentos localmente (formato e dígitos verificadores mod-11) antes de encaminhar a consulta ao serviço externo.

Base URL (Docker): `http://localhost:8081`

Base URL (local): `http://localhost:8080`

Autenticação

Todas as rotas (exceto `/health`) exigem o header:

```
Authorization: Bearer <ApiToken>
```

O token é configurado via variável de ambiente `ApiToken`.

Endpoints

GET `/health`

Verifica se o serviço está no ar. **Não requer autenticação.**

Resposta 200:

```
{ "status": "ok" }
```

GET `/consulta/cpf/{cpf}`

Consulta os dados de um CPF na Receita Federal.

Parâmetros de Path

Parâmetro	Tipo	Descrição
cpf	string	CPF com 11 dígitos. Aceita formato 000.000.000-00 ou somente dígitos 00000000000

Resposta 200 — Sucesso

```
{
  "cpf": "529.982.247-25",
  "nome": "FULANO DE TAL",
  "situacao": "REGULAR"
}
```

Campo	Tipo	Descrição
cpf	string	CPF formatado (000.000.000-00)
nome	string	Nome completo conforme Receita Federal
situacao	string	Situação cadastral (ex: REGULAR , IRREGULAR , SUSPENSA)

GET /consulta/cnpj/{cnpj}

Consulta os dados de um CNPJ na Receita Federal.

Parâmetros de Path

Parâmetro	Tipo	Descrição
cnpj	string	CNPJ com 14 dígitos. Aceita formato 00.000.000/0000-00 ou somente dígitos

Nota: CNPJs formatados contêm / na URL (ex: /consulta/cnpj/11.222.333/0001-81). A rota usa parâmetro catch-all para lidar com isso corretamente.

Resposta 200 — Sucesso

```
{
  "cnpj": "11.222.333/0001-81",
  "nome": "EMPRESA EXEMPLO LTDA",
  "fantasia": "EXEMPLO",
  "situacao": "ATIVA",
  "abertura": "01/01/2000"
}
```

Campo	Tipo	Descrição
cnpj	string	CNPJ formatado (00.000.000/0000-00)
nome	string	Razão social
fantasia	string	Nome fantasia
situacao	string	Situação cadastral (ex: ATIVA , BAIXADA , INAPTA)
abertura	string	Data de abertura no formato DD/MM/AAAA

Tabela de Códigos de Resposta

HTTP Status	Situação
200 OK	Consulta realizada com sucesso
400 Bad Request	CPF/CNPJ com formato inválido ou dígitos verificadores incorretos
401 Unauthorized	Token ausente, sem prefixo Bearer ou valor incorreto
404 Not Found	CPF/CNPJ não encontrado na base da Receita Federal
503 Service Unavailable	API externa indisponível ou timeout de 10s atingido
500 Internal Server Error	Erro interno inesperado

Formato dos erros (4xx / 5xx)

```
{ "error": "mensagem descritiva do erro" }
```

Exemplos com curl

CPF

```
# Consulta com CPF em dígitos puros
curl -s \
  -H "Authorization: Bearer supersecret123" \
  http://localhost:8081/consulta/cpf/52998224725 | jq .

# Consulta com CPF formatado
curl -s \
  -H "Authorization: Bearer supersecret123" \
  "http://localhost:8081/consulta/cpf/529.982.247-25" | jq .

# CPF inválido → 400
curl -s \
  -H "Authorization: Bearer supersecret123" \
  http://localhost:8081/consulta/cpf/00000000000 | jq .

# Sem token → 401
curl -s http://localhost:8081/consulta/cpf/52998224725 | jq .

# Token errado → 401
curl -s \
  -H "Authorization: Bearer token-errado" \
  http://localhost:8081/consulta/cpf/52998224725 | jq .
```

CNPJ

```
# Consulta com CNPJ formatado (notar as aspas por causa da barra)
curl -s \
  -H "Authorization: Bearer supersecret123" \
  "http://localhost:8081/consulta/cnpj/11.222.333/0001-81" | jq .

# Consulta com CNPJ em dígitos puros
curl -s \
  -H "Authorization: Bearer supersecret123" \
  http://localhost:8081/consulta/cnpj/11222333000181 | jq .

# CNPJ inválido → 400
curl -s \
  -H "Authorization: Bearer supersecret123" \
  "http://localhost:8081/consulta/cnpj/00.000.000/0000-00" | jq .
```

Health check

```
curl -s http://localhost:8081/health
```

Exemplos com Postman

Configurar Environment

Variável	Valor
base_url	http://localhost:8081
token	supersecret123

Consultar CPF

- **Method:** GET
- **URL:** {{base_url}}/consulta/cpf/52998224725
- **Headers:** Authorization: Bearer {{token}}

Consultar CNPJ

- **Method:** GET
- **URL:** {{base_url}}/consulta/cnpj/11222333000181
- **Headers:** Authorization: Bearer {{token}}

Testar 401

- **Method:** GET
- **URL:** {{base_url}}/consulta/cpf/52998224725
- (sem header Authorization)

Testar 400

- **Method:** GET
- **URL:** {{base_url}}/consulta/cpf/11111111111
- **Headers:** Authorization: Bearer {{token}}

Observações sobre a ReceitaWS

- A [ReceitaWS](#) é um serviço externo pago. **Sem token**, a API retorna **404** mesmo para documentos válidos.
- Configure `ReceitaWsToken` e/ou `ReceitaWsCnpjToken` no `.env` para habilitar consultas reais.
- O timeout de resposta da ReceitaWS é de **10 segundos**; após isso, a API retorna **503**.

- A validação local (formato e dígitos verificadores) ocorre **antes** de qualquer chamada externa; um CPF/CNPJ inválido retorna **400** sem consumir cota da ReceitaWS.

Documentação de Testes

Visão Geral

O projeto `CpfApi.Tests` contém uma suite xUnit (.NET 8) com **60 testes** cobrindo validators, services, middleware e integração de endpoints. Nenhuma dependência externa é necessária: o `HttpClient` e os serviços de negócio são mockados em memória.

Estrutura

```
CpfApi.Tests/  
├─ CpfApi.Tests.csproj  
├─ Validators/  
│   ├─ CpfValidatorTests.cs      (12 testes)  
│   └─ CnpjValidatorTests.cs    (10 testes)  
├─ Services/  
│   ├─ CpfServiceTests.cs       (7 testes)  
│   └─ CnpjServiceTests.cs      (6 testes)  
├─ Middleware/  
│   └─ BearerAuthMiddlewareTests.cs (6 testes)  
└─ Integration/  
    └─ ApiIntegrationTests.cs    (8 testes + ApiFactory)
```

Total: 60 testes | 0 falhas

Como Rodar

```
# A partir da raiz do repositório  
cd CpfApi.Tests  
dotnet test  
  
# Com output detalhado  
dotnet test --verbosity normal  
  
# Filtrar por categoria  
dotnet test --filter "FullyQualifiedName~Validators"  
dotnet test --filter "FullyQualifiedName~Services"  
dotnet test --filter "FullyQualifiedName~Middleware"  
dotnet test --filter "FullyQualifiedName~Integration"
```

Testes por Categoria

Validators (22 testes)

Testes puramente unitários das funções `Strip`, `Format` e `Validate`.

CpfValidatorTests (12 testes)

Teste	Descrição
<code>Validate_ValidCpf_ReturnsTrue</code>	CPF válido nos formatos formatado e sem pontuação
<code>Validate_AllSameDigits_ReturnsFalse</code>	Rejeita 111.111.111-11, 000..., 999...
<code>Validate_WrongLength_ReturnsFalse</code>	Rejeita strings com 0, 10 e 12 dígitos
<code>Validate_InvalidCheckDigit_ReturnsFalse</code>	Rejeita CPFs com 1º ou 2º dígito verificador errado
<code>Strip_FormattedCpf_ReturnsDigitsOnly</code>	529.982.247-25 → 52998224725
<code>Strip_AlreadyStripped_ReturnsSameValue</code>	Idempotente em strings sem formatação
<code>Format_RawDigits_ReturnsFormatted</code>	52998224725 → 529.982.247-25
<code>Format_AlreadyFormatted_ReturnsFormatted</code>	Idempotente em string já formatada
<code>Format_WrongLength_ReturnsInputUnchanged</code>	Retorna entrada original se não tiver 11 dígitos

CnpjValidatorTests (10 testes)

Teste	Descrição
<code>Validate_ValidCnpj_ReturnsTrue</code>	CNPJ válido em três formatos diferentes
<code>Validate_AllSameDigits_ReturnsFalse</code>	Rejeita 00000000000000, 111..., 999...
<code>Validate_WrongLength_ReturnsFalse</code>	Rejeita strings com 13 e 15 dígitos
<code>Validate_InvalidCheckDigit_ReturnsFalse</code>	Rejeita CNPJs com 1º ou 2º DV errado
<code>Strip_FormattedCnpj_ReturnsDigitsOnly</code>	11.222.333/0001-81 → 11222333000181
<code>Strip_AlreadyStripped_ReturnsSameValue</code>	Idempotente
<code>Format_RawDigits_ReturnsFormatted</code>	11222333000181 → 11.222.333/0001-81
<code>Format_WrongLength_ReturnsInputUnchanged</code>	Retorna entrada original se não tiver 14 dígitos

Services (13 testes)

Testes do `CpfService` e `CnpjService` usando `HttpMessageHandler` customizado — nenhum servidor real é chamado.

`CpfServiceTests` (7 testes)

Teste	Comportamento verificado
<code>ConsultarAsync_SuccessResponse_ReturnsCpfResponse</code>	HTTP 200 com JSON válido → retorna <code>CpfResponse</code> com CPF formatado
<code>ConsultarAsync_NotFound_ThrowsCpfNotFoundException</code>	HTTP 404 → lança <code>CpfNotFoundException</code>
<code>ConsultarAsync_ServerError_ThrowsCpfServiceUnavailableException</code>	HTTP 500 → lança <code>CpfServiceUnavailableException</code>
<code>ConsultarAsync_ErrorStatusInBody_ThrowsCpfServiceUnavailableException</code>	JSON com "status": "ERROR" → lança exceção com a mensagem no campo <code>message</code>
<code>ConsultarAsync_NetworkError_ThrowsCpfServiceUnavailableException</code>	<code>HttpRequestException</code> → lança <code>CpfServiceUnavailableException</code>
<code>ConsultarAsync_WithToken_AppendsTokenToUrl</code>	Com <code>ReceitaWsToken</code> configurado → URL inclui <code>?token=...</code>
<code>ConsultarAsync_WithoutToken_DoesNotAppendToken</code>	Sem token configurado → URL não inclui <code>token</code>

CnpjServiceTests (6 testes)

Teste	Comportamento verificado
ConsultarAsync_SuccessResponse_ReturnsCnpjResponse	HTTP 200 → retorna CnpjRes com todos os campos mapeados
ConsultarAsync_NotFound_ThrowsCnpjNotFoundException	HTTP 404 → lança CnpjNotFoundException
ConsultarAsync_ServerError_ThrowsCnpjServiceUnavailableException	HTTP 500 → lança CnpjServiceUnavailableException
ConsultarAsync_ErrorStatusInBody_ThrowsCnpjServiceUnavailableException	JSON com "status": "ERROR" → mensagem de erro propagada
ConsultarAsync_NetworkError_ThrowsCnpjServiceUnavailableException	HttpRequestException → lança CnpjServiceUnavailableException
ConsultarAsync_WithToken_AppendsTokenToUrl	ReceitaWsCnpjToken configurado → ?token=... na URL

Middleware (6 testes)

Testes do BearerAuthMiddleware com DefaultHttpContext em memória.

Teste	Comportamento verificado
InvokeAsync_ValidToken_CallsNext	Token correto → delega para o próximo handler (status 200)
InvokeAsync_MissingAuthHeader_Returns401	Sem header Authorization → 401
InvokeAsync_WrongToken_Returns401	Token diferente do configurado → 401, handler não é chamado
InvokeAsync_BearerPrefixMissing_Returns401	Header sem prefixo Bearer → 401
InvokeAsync_HealthPath_SkipsAuth	Rota /health → middleware ignorado, handler chamado sem token
InvokeAsync_EmptyConfiguredToken_Returns401	ApiToken vazio na configuração → 401 (serviço mal configurado)

Integration (8 testes)

Testes de ponta a ponta usando `WebApplicationFactory<Program>`. O `ICpfService` e o `ICnpjService` são substituídos por implementações fake via `ConfigureTestServices` — nenhuma chamada HTTP externa ocorre.

Teste	Endpoint	Status esperado
<code>Health_ReturnsOk</code>	<code>GET /health</code>	200
<code>ConsultaCpf_WithoutToken_Returns401</code>	<code>GET /consulta/cpf/529...</code> (sem token)	401
<code>ConsultaCpf_InvalidCpf_Returns400</code>	<code>GET /consulta/cpf/</code> <code>000.000.000-00</code>	400
<code>ConsultaCpf_ValidCpf_Returns200</code>	<code>GET /consulta/cpf/</code> <code>529.982.247-25</code>	200 + body correto
<code>ConsultaCpf_NotFound_Returns404</code>	CPF que o fake service rejeita	404
<code>ConsultaCnpj_WithoutToken_Returns401</code>	<code>GET /consulta/cnpj/</code> <code>11.222.333/0001-81</code> (sem token)	401
<code>ConsultaCnpj_InvalidCnpj_Returns400</code>	<code>GET /consulta/cnpj/</code> <code>00.000.000/0000-00</code>	400
<code>ConsultaCnpj_ValidCnpj_Returns200</code>	<code>GET /consulta/cnpj/</code> <code>11.222.333/0001-81</code>	200 + body correto

Dependências de Teste

Pacote	Versão	Uso
<code>xunit</code>	2.7.0	Framework de testes
<code>xunit.runner.visualstudio</code>	2.5.7	Runner para <code>dotnet test</code>
<code>Microsoft.NET.Test.Sdk</code>	17.9.0	SDK de testes
<code>Microsoft.AspNetCore.Mvc.Testing</code>	8.0.0	<code>WebApplicationFactory</code> para testes de integração

Sem Moq ou outras bibliotecas de mock: todos os doubles de teste são implementações `file` - scoped no próprio arquivo de teste.